

ASP. NET Web API 2.2

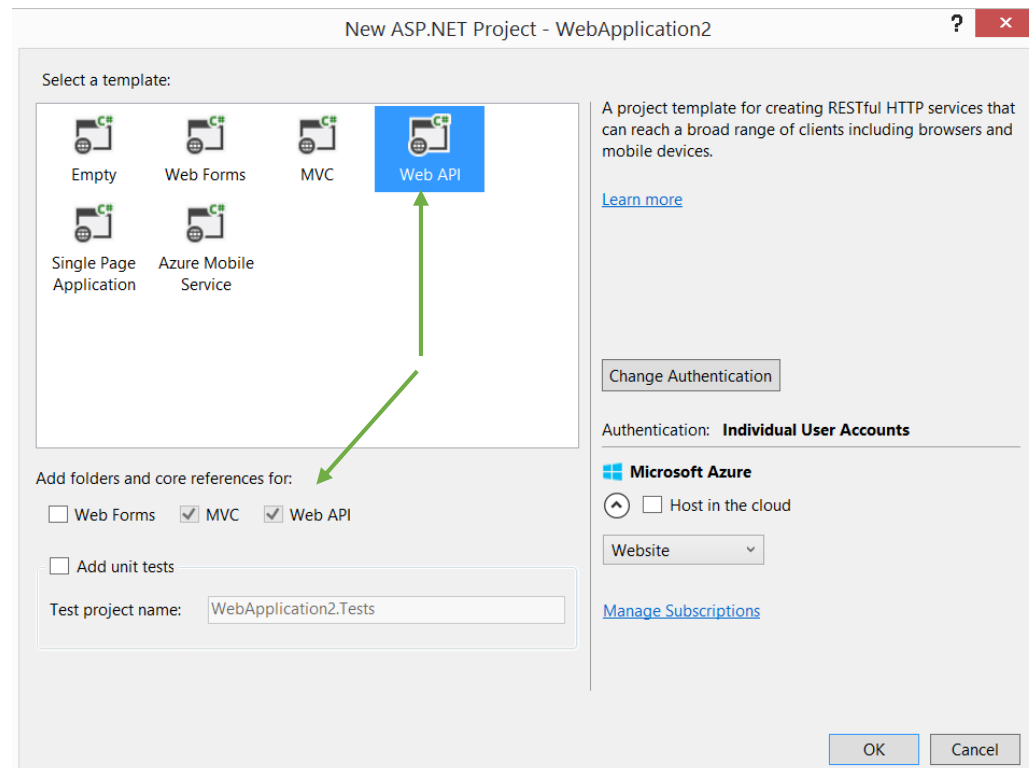


Agenda

- SOAP? REST? OData?
- Web API Framework
- Hosting
- Security
- Consuming
- Clients mit JavaScript und C#

ASP.NET Web API

- Ein mögliches Backend für Web und Desktop Apps
- Erstellung von HTTP-basierten Services leicht möglich
- Ideal für RESTful APIs



REST

Representational State Transfer



Einführung REST

- **Die 4 Schichten des TCP/IP Referenzmodells**

Schicht	Name	Beschreibung
1	Netzzugangsschicht	Beschreibt Techniken zur Datenübertragung z.B. Verwendung der Netzwerkkarte – Protokolle: <i>Ethernet, FDDI, PPP, 802.11</i>
2	Internetschicht	Zuständig für die Weitervermittlung von Paketen – Protokoll: <i>IP (Internet Protocol)</i>
3	Transportschicht	Beschreibt wie Daten von einem Punkt zu einem anderen transportiert werden – Protokolle: TCP, UDP
4	Anwendungsschicht	Beschreibt die Bedeutung und das Format der transportierten Daten – Protokoll: HTTP

Einführung REST

„...the most efficient network request is one that doesn't use the network.“

- *Roy Fielding*

Einführung REST

- **Was ist REST ?**
 - Paradigma der Softwareentwicklung
 - Kein Protokoll wie HTTP oder SOAP



Einführung REST

- **Welche Eigenschaften muss ein RESTful Service haben ?**

- **Client Server Architektur**

- Eine einheitliche Schnittstelle trennt Client und Server.

- **Statelessness**

- Zustandslose Interaktion zwischen Client und Server.

- **Cacheability**

- Nachrichten vom Server (Responses) müssen sich selbst als cacheable oder nicht cacheable auszeichnen, damit der Client keine veralteten Daten zwischenspeichert.

Einführung REST

- **Welche Eigenschaften muss ein RESTful Service haben ?**

- **Layered System**

- Der Client hat keine Information darüber, ob er direkt mit dem finalen Server verbunden ist.

- **Uniform interface**

- Jeder Dienst, jede Methode oder jeder Status besitzt eine eindeutige Adresse.

- **Code on demand (*optional*)**

- Server kann die Funktionalität des Clients anpassen, indem ausführbarer Code mitgeliefert wird.

Einführung REST

- REST Operationen mit Hilfe von HTTP Verben

HTTP Methode	CRUD action	Beschreibung
GET	Read	Gibt einen einzelnen oder eine Sammlung von Einträgen zurück.
POST	Create	Erzeugen eines neuen Eintrags.
PUT	Update	Update (Aktualisierung) eines bestehenden Eintrags.
DELETE	Delete	Löschen eines Eintrags.
PATCH	Update	Teile eines Eintrags werden geändert / aktualisiert
Head	N/A	Liest die Message Header anhand der URI aus

Einführung REST

- Die wichtigsten HTTP Status Klassen
- HTTP-Status als Antwort möglich

HTTP Status	Beschreibung
1xx	Informativ
2xx	Erfolgreich (200 = OK, 201 = Created)
3xx	Umleitung
4xx	Client Error (400 = Bad Request, 404 = Not Found)
5xx	Server Error

Einführung REST

- **Schlüsselemente eines REST Service**

- **Uniform Resource Identifier (*URI*)**

- Lokalisiert eine spezifische Ressource über eine Zugriffsmethode (bspw. HTTP) in einem Netzwerk.

- **Links**

- Server stellt bearbeitbare Ressource mit Hilfe eines Hypermedia Links zur Verfügung. Konzept: *Hypermedia as the Engine of Application State (HATEOAS)*.

- **Uniform Interface**

- Service verwendet HTTP Verben, um auf adressierbare Ressourcen über eine URI zugreifen zu können.

Einführung REST

- **Schlüsselemente eines REST Service**

- **Stateless Communication**

- Zustandslose Kommunikation: Client muss Informationen im Request zur Verfügung stellen, damit Server eine entsprechende Response liefern kann.

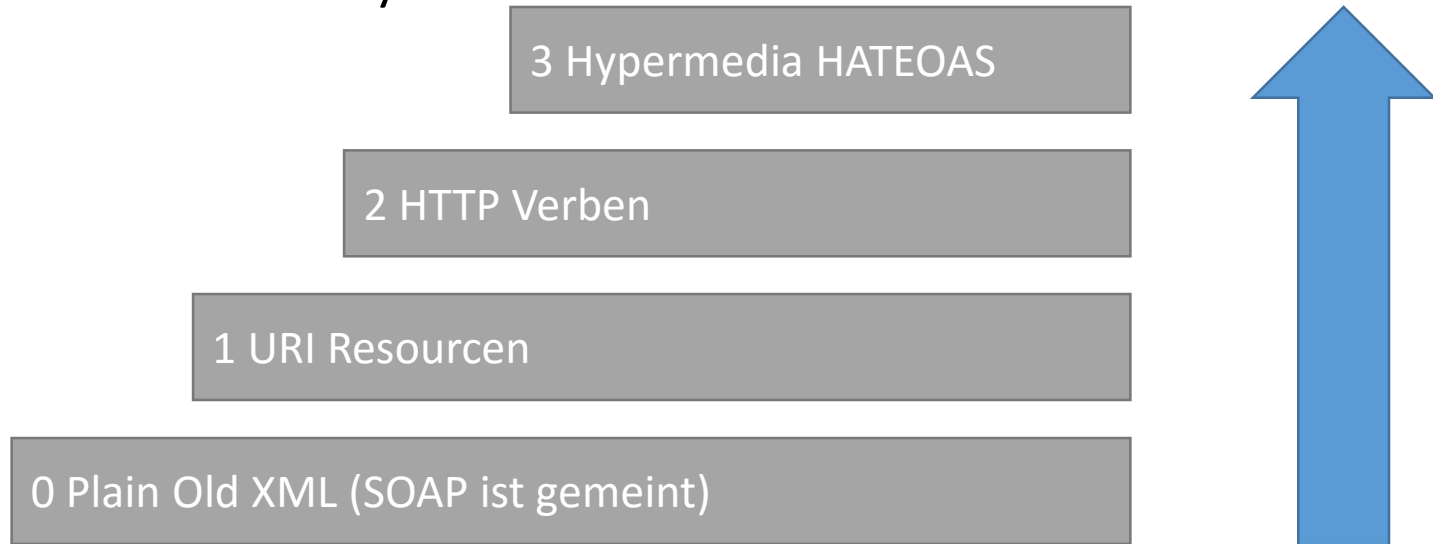
- **Mehrfache Repräsentationen einer Ressource**

- Jeder Service kann verschiedene „Repräsentationen“ einer Datenquelle liefern, wie beispielsweise JSON oder XML.



Einführung REST

- Richardson Maturity Model



Hypermedia as the Engine of Application State

Einführung REST

- Original Dissertation über REST



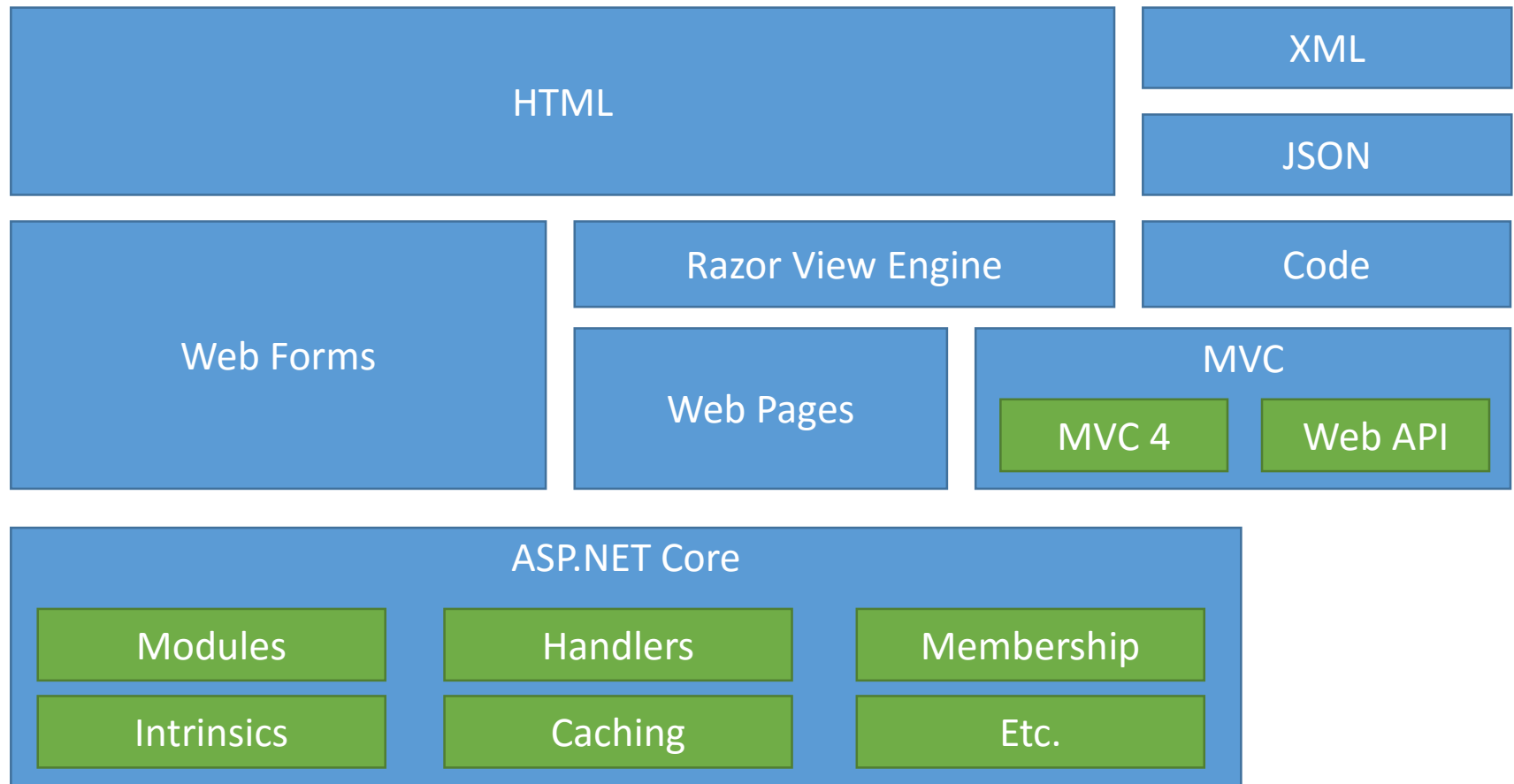
<http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Web API 2.2

Das Microsoft Framework zur Erstellung von REST Services

Grundlagen ASP.NET Web API

- **Web API ist ein Teil von ASP.NET**



Grundlagen ASP.NET Web API

- **Was ist die ASP.NET Web API ?**
 - **Eine Bibliothek zur Erstellung von HTTP Services**
 - HTTP ermöglicht eine hohe Skalierbarkeit
 - Leichtgewicht aus dem Hause Microsoft
 - **Entwickelt, um REST Services leicht zu erstellen**
 - **Perfekt für heterogene Gerät/OS Szenarien**
 - Keine Probleme im Bezug auf Versionierung wie bei SOAP
 - **Mehrwert für das ASP.NET MVC 4 Modell**



Grundlagen ASP.NET Web API

- **Was kann die ASP.NET Web API?**
 - **Keine Bindung an Webanwendungen**
 - **Konvention über Konfiguration**
 - Verwendung von HTTP Methoden und HTTP Status Codes
 - **Content Negotiation**
 - „Inhaltsvereinbarung“ zwischen Client-Server
 - Die Web API gibt das geforderte Ergebnis serialisiert in der Form aus, die vom Client als akzeptierte Form angegeben wird (XML, JSON, ...).
 - Steuerung über den Request Header: Client sagt was er versteht
 - **Caching**
 - Unterstützt Standard HTTP Caching Mechanismus



Grundlagen ASP.NET Web API

- **Wo kann die ASP.NET Web API verwendet werden?**
 - Backend für Mobile Clients
 - Backend für Single Page Applikation
 - Backend für Desktop Anwendungen
 - Kann ASP.NET MVC und Web Forms ergänzen



Grundlagen ASP.NET und Web API

- **Web API VS. WCF**

- WCF ist ein Remote-Procedure Call und ruft Funktion auf entfernter Maschine auf
- Web API ist ein Rest Call und liefert immer das gleiche Ergebnis
- Nachteil WCF: Contract First Design und setzt ein Interface voraus
- Vorteil Web API: Ungebunden an UI, Dokument werden geholt
- Für HTTP Services ist Web API zu bevorzugen
- Bei TCP, Named Pipes, UDP, etc. scheidet Web API jedoch aus

Grundlagen ASP.NET und Web API

- **Wie kann Web API 2.2 installiert werden?**
 - Download NuGet Packages
 - ASP.NET Web API 2.1 ab Visual Studio 2013 Update 2
 - .NET 4.5+ erforderlich
 - Nützliche Links
 - Beispiele: <http://www.asp.net/web-api>
 - Sourcecode: <http://aspnetwebstack.codeplex.com/>

Web API und HTTP

- ‚Hello PPEDV‘ mit Web API 2.2
- Visual Studio → File → New Project
- Visual C# → Web → ASP.NET Web Application
- Vorlage: Web API, Change Authentication: No Authentication
- In der Datei Values Controller die GET Methode auf Hello PPEDV anpassen
- → CODE Snippets einbauen!!
- Fertige Methode pasten

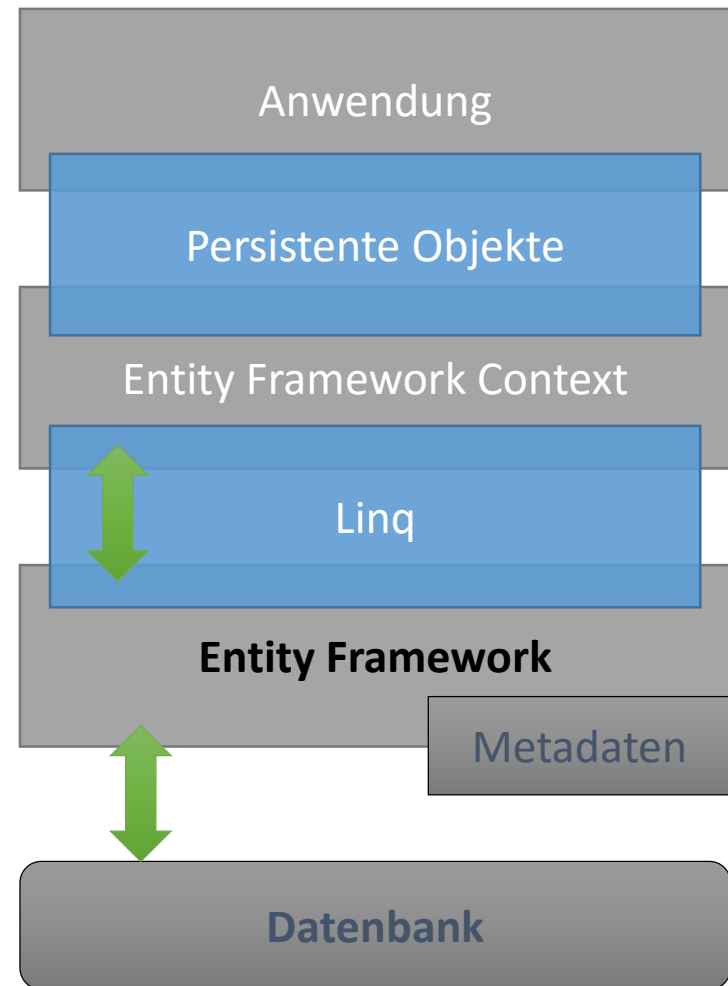


Demo

Web API und HTTP

• Exkurs Entity Framework

- O/R Mapping Framework
- Übersetzt LINQ in SQL
- Abstrahiert ADO.NET
- Ermöglicht es, Daten nur bei Bedarf zu laden (*lazy loading*)
- Der Einsatz von O/R Mappern spart viel Zeit!



Web API und HTTP

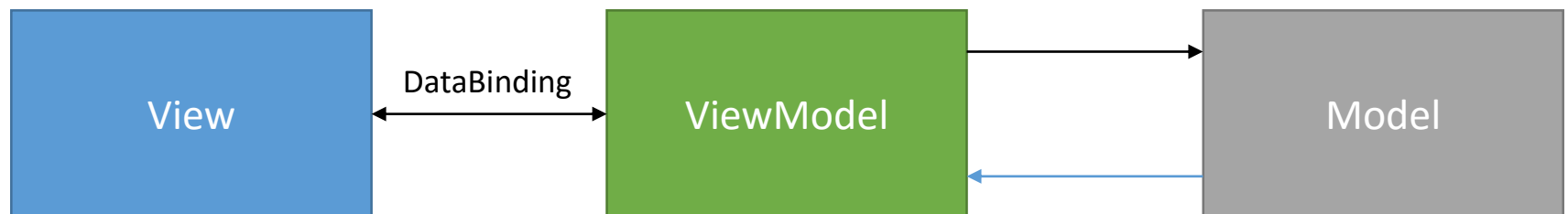
- **Exkurs LINQ**

- Language Integrated Query
- Abfragesprachen auf Objektmengen
- Direkter Bestandteil von C# und VB.NET
- Bietet die Möglichkeit, Abfragen abstrakt zu beschreiben, die später durch entsprechende Provider in konkrete Abfragen für die entsprechenden Zielsysteme überführt werden.
- 101 LINQ Beispiele:
 - <https://code.msdn.microsoft.com/101-LINQ-Samples-3fb9811b>

Web API und HTTP

- **Model – View – ViewModel (MVVM)**

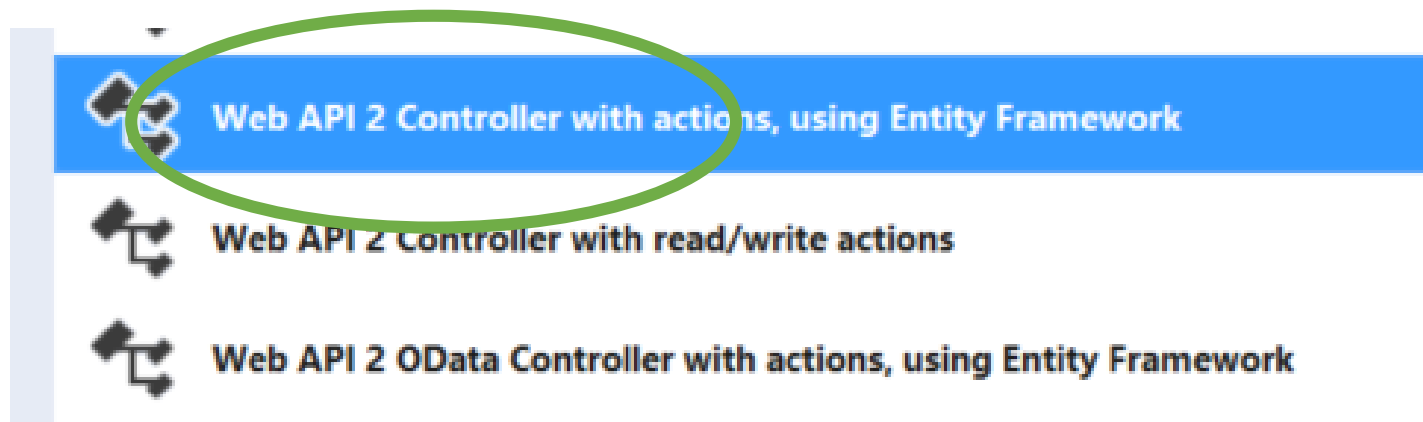
- View
 - „Darstellung“ - Inline-Code und Code-Behind
- ViewModel
 - „Vermittler“ Status und Operationen
- Model
 - Datenbasis



Web API und HTTP

- **API Controller**

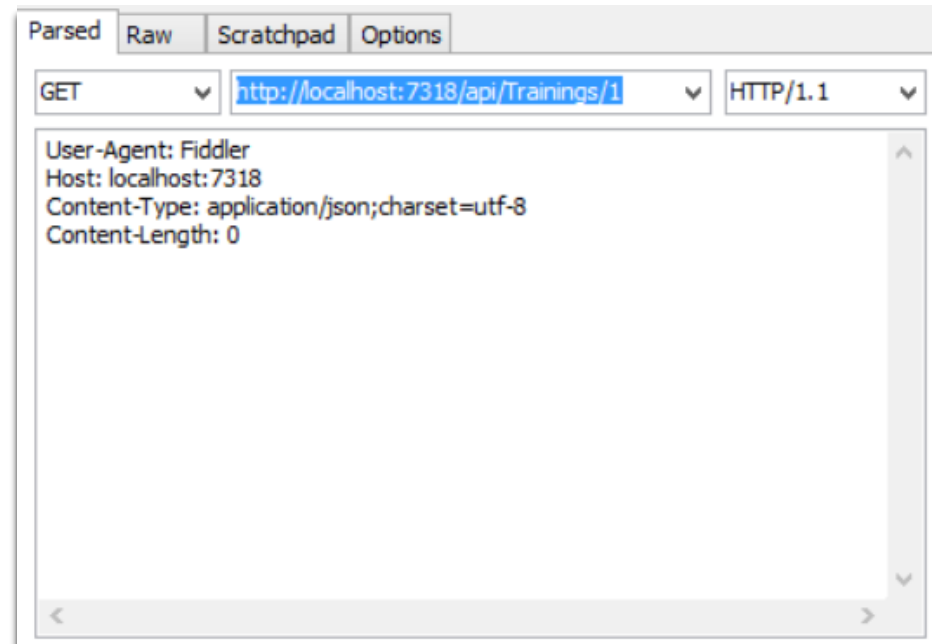
- Der Controller ist die Klasse, welche sämtliche Methoden enthält, die eine Anfrage (Request) entgegennehmen und beantworten können.
- Implementierung von: GET, POST, PUT, DELETE, ...



Web API und HTTP

- **Rest Tool - Fiddler**

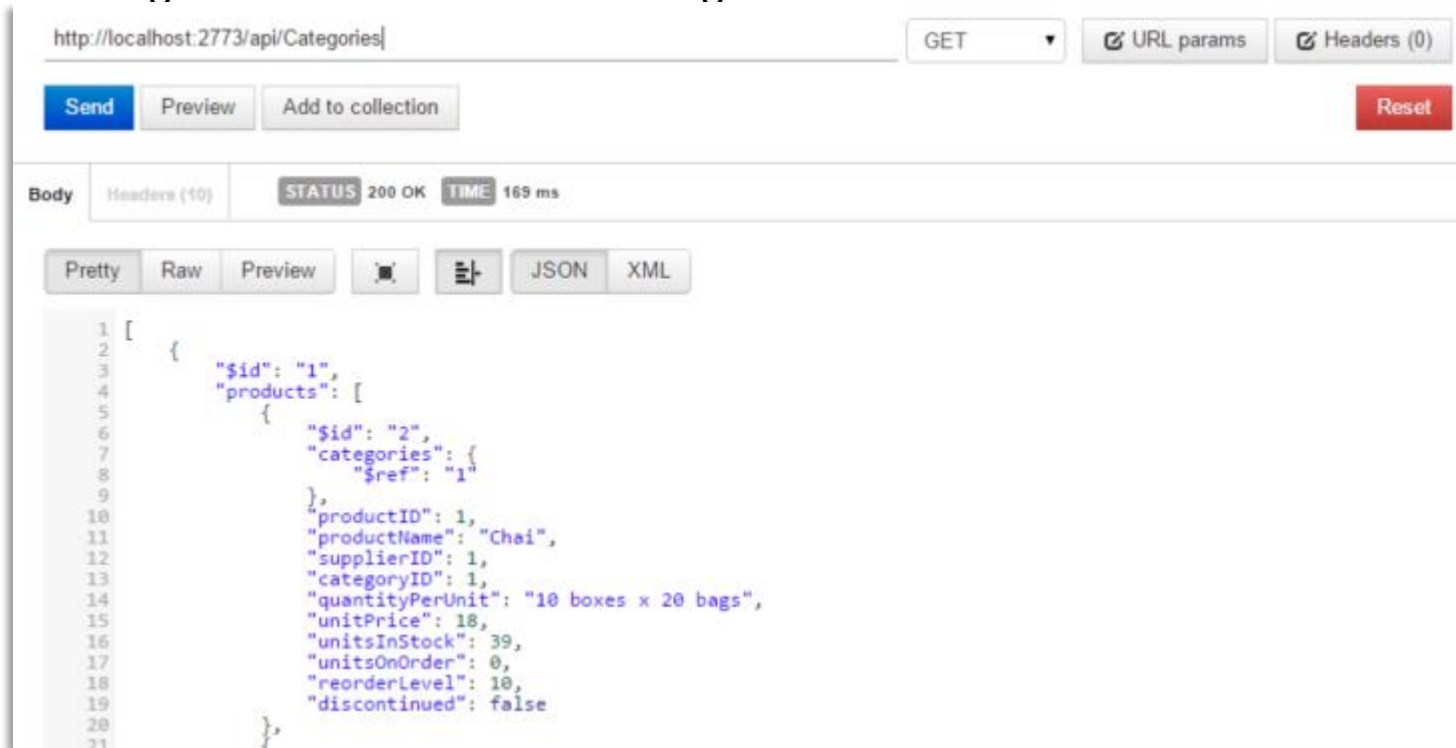
- <http://www.telerik.com/fiddler>
- Web Debugging
- Web Session Manipulation
- Performance Testing
- Security Testing
- HTTP/HTTPS Traffic Recording
- ...



Web API und HTTP

- **Rest Tool im Browser**

- Google Chrome mit Postman Plugin



0 Code Beispiel

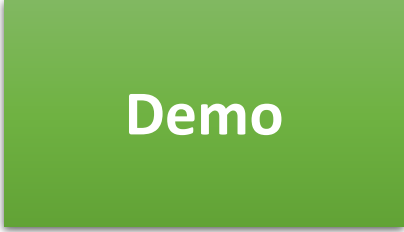
Web API mit Hilfe von Entity Framework (Version 6.1)



Web API und HTTP

- **ZeroCodeBeispiel**

- No Authentication
- CodeFirst from Database → Northwind
- (localdb)\v11.0
- CustomersSet
- Anpassen der Ausgabe



Demo

Web API und HTTP

The screenshot shows a web client interface with the following components:

- URL bar:** `http://localhost:1800/api/Customers/`
- Method:** `GET` (highlighted with a green circle)
- Buttons:** `Send`, `Preview`, `Add to collection`, `URL params`, `Headers (0)`, `Reset`
- Status:** `STATUS 200 OK`, `TIME 176 ms`
- Response Body:** A JSON array of two customer objects.

The JSON response is displayed in a code editor with the following content:

```
1 [
2   {
3     "CustomerID": "ALFKI",
4     "CompanyName": "Alfreds Futterkiste",
5     "ContactName": "Maria Anders",
6     "ContactTitle": "Sales Representative",
7     "Address": "Obere Str. 57",
8     "City": "Berlin",
9     "Region": null,
10    "PostalCode": "12209",
11    "Country": "Germany",
12    "Phone": "030-0074321",
13    "Fax": "030-0076545"
14  },
15  {
16    "CustomerID": "ANATR",
17    "CompanyName": "Ana Trujillo Emparedados y helados",
18    "ContactName": "Ana Trujillo",
19    "ContactTitle": "Owner",
20    "Address": "Avda. de la Constitución 2222",
21    "City": "México D.F.",
22    "Region": null,
23    "PostalCode": "05021",
24    "Country": "Mexico",
25    "Phone": "(5) 555-4729",
26    "Fax": "(5) 555-3745"
27  }
28 ]
```


Web API und HTTP

The screenshot displays a REST client interface with the following components:

- URL Bar:** Contains the URL `http://localhost:1800/api/Customers/ALFKI`. The word **KEY** is written below the `ALFKI` segment, which is circled in green.
- Method:** A dropdown menu shows `GET`, which is also circled in green. The word **GET** is written below the dropdown.
- Buttons:** `URL params` and `Headers (0)` are on the right. `Send`, `Preview`, and `Add to collection` are below the URL bar. A red `Reset` button is on the far right.
- Response Section:**
 - Body:** The selected tab.
 - Headers (10):** A tab for viewing headers.
 - STATUS:** `200 OK`
 - TIME:** `167 ms`
- Format Buttons:** `Pretty`, `Raw`, `Preview`, and icons for `JSON` and `XML`.
- Response Body:** A JSON object displayed in a code editor:

```
1 {  
2   "CustomerID": "ALFKI",  
3   "CompanyName": "Alfreds Futterkiste",  
4   "ContactName": "Maria Anders",  
5   "ContactTitle": "Sales Representative",  
6   "Address": "Obere Str. 57",  
7   "City": "Berlin",  
8   "Region": null,  
9   "PostalCode": "12209",  
10  "Country": "Germany",  
11  "Phone": "030-0074321",  
12  "Fax": "030-0076545",  
13 }
```

Web API und HTTP

- **Content Negotiation**

- „Inhaltsvereinbarung“ zwischen Client-Server
- XML oder JSON im **Accept** header
- Wird ein Format nicht gefunden: **406 Not Acceptable**
- XML basiert auf der ASP.NET Klasse **XmlMediaTypeFormatter**

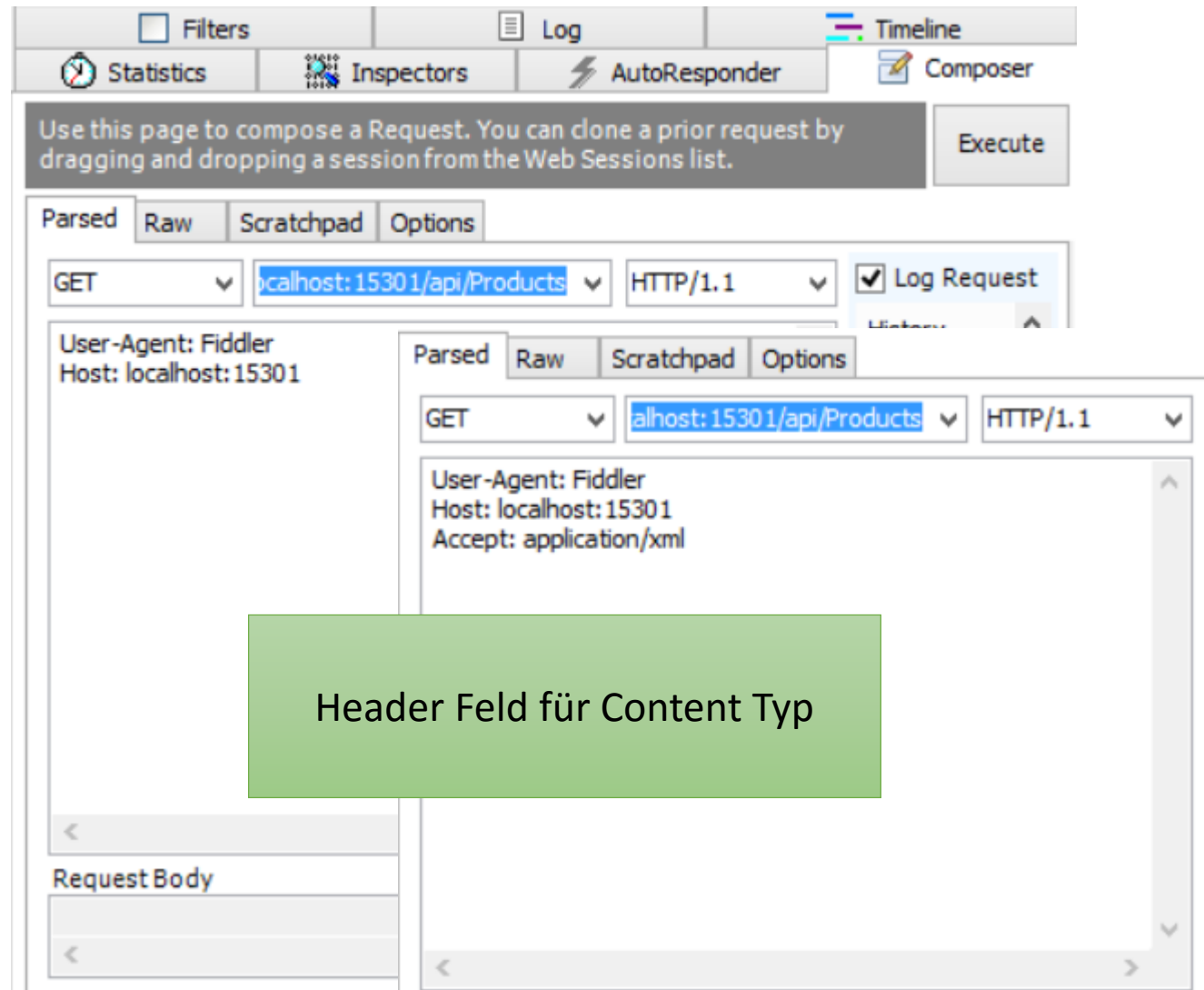
- **Accept Header**

- Accept: application/xml
- Accept: application/json; charset=utf-8
- Für POST und PUT „Content-Type“ statt „Accept“ Attribut



Web API und HTTP

- Fiddler



Web API und HTTP

- Fiddler

The screenshot displays the Fiddler web proxy tool interface. The top menu bar includes Composer, Filters, Log, and Timeline. Below it are Statistics, Inspectors, and AutoResponder. The main window is divided into several panes. On the left, the 'Request Headers' pane shows a GET request to /api/Products. The 'Client' section indicates the User-Agent is Fiddler, and the 'Transport' section shows the Host is localhost:15301. The 'JSON' pane shows a tree view of the response body, which is an array of products. The 'XML' pane shows the same data in XML format, with a tree view of the response body. A green box with the text 'JSON oder XML im Inspector' is overlaid on the XML pane. The status bar at the bottom indicates 'JSON parsing completed.'

JSON oder XML im Inspector

Web API und HTTP

- **POST Request / Daten schreiben**

The screenshot shows a REST client interface with the following elements:

- URL:** `http://localhost:1800/api/Customers/`
- Method:** **POST** (circled in green)
- Host:** `localhost:1800`
- Content-Type:** `application/json; charset=utf-8` (circled in green)
- Header:** A table with columns "Header" and "Value".
- Manage presets** button.
- Formatters:** `form-data`, `x-www-form-urlencoded`, `raw`, and `Text` (selected).
- JSON Body:** A JSON object (circled in green) with the following structure:

```
1 {
2   "CustomerID": "NEWKI",
3   "CompanyName": "Neumann GmbH",
4   "ContactName": "Charlotte Musterfrau",
5   "ContactTitle": "Verkauf",
6   "Address": "Weg 7",
7   "City": "Burghausen",
8   "Region": null,
9   "PostalCode": "12345",
10  "Country": "Germany",
11  "Phone": "030-0074321",
12  "Fax": "030-0076545"
13 }
```
- Buttons:** `Send`, `Preview`, `Add to collection`, and `Reset`.

Datensatz

Web API und HTTP

- PUT Request / Daten updaten

http://localhost:1800/api/Customers/NEWKI PUT URL params Headers (2)

Host localhost:1800 PUT

Content-Type application/json;charset=utf-8

Header	Value
--------	-------

Manage presets

form-data x-www-form-urlencoded raw Text

```
1 {
2   "CustomerID": "NEWKI",
3   "CompanyName": "Neumann GmbH",
4   "ContactName": "Manfred Mustermann",
5   "ContactTitle": "Verkauf",
6   "Address": "Weg 10",
7   "City": "München",
8   "Region": null,
9   "PostalCode": "12345",
10  "Country": "Germany",
11  "Phone": "030-0074321",
12  "Fax": "030-0076545"
13 }
```

Datensatz

Send Preview Add to collection Reset

Web API und HTTP

- **DELETE Request / Daten löschen**

The screenshot shows a REST client interface with the following elements:

- URL bar:** Contains the URL `http://localhost:1800/api/Customers/NEWKI` and a dropdown menu set to `DELETE`. To the right are buttons for `URL params` and `Headers (2)`.
- Host field:** Labeled **Host** with the value `localhost:1800`. Above it, the word **KEY** is written, and to its right, the word **DELETE** is written. A close button (X) is next to the field.
- Content-Type field:** Labeled **Content-Type** with the value `application/json; charset=utf-8`. A close button (X) is next to the field.
- Header field:** Labeled **Header** with the value `Value`.
- Manage presets** button.
- Form tabs:** `form-data`, `x-www-form-urlencoded`, `raw`, and `Text` (selected).
- Request body:** A large text area containing the text `1 |`.
- Buttons at the bottom:** `Send` (blue), `Preview` (grey), `Add to collection` (grey), and `Reset` (red).

Formatierung

Erweiterte Techniken



Web API Erweiterte Techniken

- **Formatting von Ressourcen**

- Formatter Klassen können :
- Daten lesen und schreiben
- Konvertieren von einem Format auf ein anderes

Members	Funktion
SupportedMediaType	Collection von Media Types, die vom Formatter unterstützt werden.
CanReadType	Boolean, gibt Auskunft darüber, ob der Formatter den Datentyp lesen kann.
CanWriteType	Boolean, gibt Auskunft darüber, ob der Formatter den Datentyp schreiben kann.
ReadFromStreamAsync	Wird beim lesen des Input Streams aufgerufen. Gibt die Ressource zurück.
WriteToStreamAsyng	Aufruf beim Schreiben der Ressource in den Output Stream.

Web API und HTTP

- „Out-of-the-box“ Media Type Formatters in ASP.NET

Media Formatter	Funktion
JsonMediaTypeFormatter	Formatiert die geforderte Ressource im JSON Format
XmlMediaTypeFormatter	Formatiert die geforderte Ressource im XML Format
FormUrlEncodedMediaTypeFormatter	Formular Requests: application/x-www-form-urlencoded
JQueryMvcFormUrlEncodedFormatter	Erbt von
FormUrlEncodedMediaTypeFormatter	Und fügt Unterstützung zu JQuery hinzu

- Das Erstellen von eigenen Formattern ist in ASP.NET Web API möglich

Web API und HTTP

- **Media Formatter**
- Anpassen der JSON Serialisierung auf javaScript Namenskonvention (CamelCase)
- Prüfung des Formats mit Fiddler



Demo

Web API und HTTP

- Verzeichnis App_Start → WebApiConfig.cs

```
// Web API configuration and services
config.Formatters.JsonFormatter
    .SerializerSettings.ContractResolver =
        new CamelCasePropertyNamesContractResolver();
```



HTTP Statuscodes

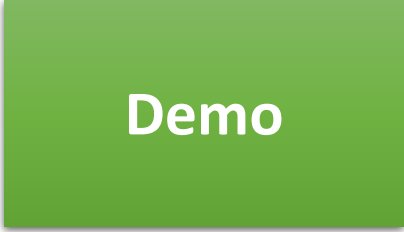
Erweiterte Techniken

Web API Erweiterte Techniken

Code	Nachricht	Bedeutung
200	OK	Erfolg, das Ergebnis der Anfrage wird in der Antwort übertragen.
201	Created	Erfolgreich bearbeitet, die Ressource wurde vor dem Senden der Antwort erstellt.
202	Accepted	Anfrage akzeptiert, wird aber erst später ausgeführt. Kein Gelingen garantiert
301	Moved Permanently	Redirect, die angeforderte Ressource steht unter einer anderen Adresse bereit. Die alte Adresse ist nicht länger gültig.
400	Bad Request	Fehlerhafte Anfrage
401	Unauthorized	Keine gültige Authentifizierung, Anfrage nicht möglich.
402	Payment Required	Status für zukünftige HTTP-Protokolle reserviert – „ <i>Bezahlung benötigt</i> “. Wird oft verwendet um Fehler im Validierungsprozess aufzudecken.
403	Forbidden	Client nicht berechtigt, Anfrage wurde nicht durchgeführt.
404	Not Found	Die angeforderte Ressource wurde nicht gefunden. Anfrage ohne Grund abgewiesen.
409	Conflict	Anfrage wurde unter falschen Annahmen gestellt.
500	Internal Server Error	Unerwarteter Serverfehler.

Web API und HTTP

- **Rückgabe HTTP Status Codes**
- *HTTP Action Result* seit Version 2 als Rückgabebetyp möglich!
- Anpassen des Controllers
 - Wenn GET Ressource nicht gefunden wurde, HTTP **404 Not Found** als Rückgabecode
 - POST, PUT, DELETE mit Rückgabebetyp *IHttpActionResult* anpassen
- Prüfung des Formats mit HTTP Debugger



Demo

Async und Await

Erweiterte Techniken

Web API Erweiterte Techniken

- **Asynchrone (Server) Entwicklung**

- „Reaktionsfähigkeit“ von Anwendungen verbessern
- „Nicht blockierendes“ Programmieren
- `async` und `await` seit C# 5.0
- `async` deklariert Methode, die in eigenen Thread ausgelagert wird und als Rückgabebetyp *Task* oder *void* verwenden muss
- `await` wird beim Beginn des asynchronen Codeblocks verwendet und gibt kontrolliert Thread an dieser Stelle
- Ausführung von asynchronem Code im Controller

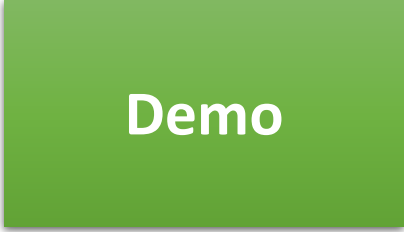
Web API Erweiterte Techniken

- **Typische Anwendungsbereiche für async. Programmierung**

Anwendungsbereich	Unterstützende APIs
Webzugriff	HttpClient, SyndicationClient
Arbeiten mit Dateien	StorageFile, StreamWriter, StreamReader, XmlReader
Arbeiten mit Bildern	MediaCapture, BitmapEncoder, BitmapDecoder
WCF-Programmierung	Synchrone und asynchrone Vorgänge

Web API Erweiterte Techniken

- Umbau des Controllers auf Asynchrone Methoden
- Prüfung des Formats mittels HTTP Debugger



Demo

Caching

Erweiterte Techniken



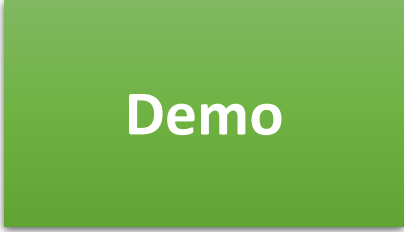
Web API Erweiterte Techniken

- **HTTP-Caching**

- Performance optimieren!
- Ein Client kann Ressourcen „cachen“ und dadurch die Zugriffszeiten verbessern.
- Der Server muss über die „Cacheability“ einer Ressource Auskunft geben. → Kann eine Ressource gecached werden?
- Ein gutes Cacheing-Plugin von Brad Wilson
 - <https://gist.github.com/bradwilson/8586562>

Web API Erweiterte Techniken

- Einbau des Caching Mechanismus z.B. mit dem Web API Plugin von Brad Wilson
- Anpassen des Controllers
- Prüfung mit Browser – HTTP Debugger ignoriert Caching



Demo

Web API Erweiterte Techniken

```
// GET: api/Customers
public IHttpActionResult GetCustomers()
{
    return
        Ok(db.Customers)
        .Cached(Cacheability.Public, maxAge:
            TimeSpan.FromMinutes(15));
}
```



Binding

Erweiterte Techniken

Web API Erweiterte Techniken

- **Objektteile zurückliefern**

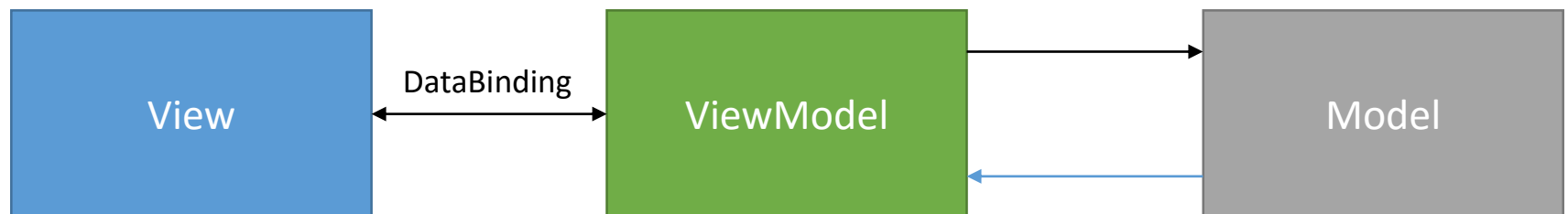
- Sinnvoll wenn weniger Daten ausgeliefert werden sollen
- Möglich über **Lambda Expression** = anonymes Objekt
- Oder über **View-Model** = „richtiges“ Objekt



Web API Erweiterte Techniken

- **Model – View – ViewModel (MVVM)**

- View
 - „Darstellung“ - Inline-Code und Code-Behind
- ViewModel
 - „Vermittler“ Status und Operationen
- Model
 - Datenbasis



Web API Erweiterte Techniken

- **Lambda Expression**

```
public async Task<IHttpActionResult> Get()
{
    using (var db = new ProductsContext ())
    {
        return Ok(await db.Products.Include(t => t.Category)
            .Select(t => new { Title = t.Name, Number = t.Id })
            .ToListAsync()).
        Cached(cacheability: Cacheability.Public,
            maxAge: TimeSpan.FromSeconds(30));
    }
}
```



Web API Erweiterte Techniken

```
// GET api/Products
public async Task<IHttpActionResult> Get()
{
    using (var db = new ProductsContext ())
    {
        return Ok(await db.Products.Include(t => t.Category)
            // Übergabe ViewModel auf Basis der akt. Entität
            .Select(t => new ProductViewModel
            { Title = t.Name, Number = t.Id })
            .ToListAsync()).
            Cached(cacheability: Cacheability.Public,
            maxAge: TimeSpan.FromSeconds(30));
    }
}
```



Web API Erweiterte Techniken

```
// GET api/Products/5
public async Task<IHttpActionResult> Get(int id)
{
    using (var db = new ProductsContext())
    {
        var result = await db.Products.Include(t => t.Category)
            .Select(t => new ProductViewModel
            { Title = t.Name, Number = t.Id })
            .SingleOrDefaultAsync(t => t.Number == id);

        if (result == null)
        {
            return NotFound();
        }

        return Ok(result);
    }
}
```



Web API Erweiterte Techniken

- **Binding**

- Aufruf einer Controller Methode mit Übergabeparametern durch die Web API
- HttpResponseMessage Put(int id, Product item) { ... }
- Id = primitive Type
- Product = Complex Type

- Complex Type aus URI: **[FromUri]**
- Complex Type aus Body: **[FromBody]**



Routing

Erweiterte Techniken

Binding, Validation, Routing

- **Routing**

- Routing erfolgt in **Routing Tabellen**
- oder auch über **Attribute** für „temporäre“ Routs
- Zuweisen von Controller und Methoden zur passenden Anfrage

- **Routing in 3 Schritten**

- Vergleichen von URI und Route Template
- Auswahl eines Controllers
- Auswahl einer Action



Binding, Validation, Routing

- **Routing Tabelle**

- App_Start → WebApiConfig.cs

```
config.Routes.MapHttpRoute(  
    name: "ProductCategoryRoute",  
    routeTemplate: "api/product/{id}/products",  
    defaults: new { controller = "Products",  
                    action = "GetByCategory" },  
    constraints: new { id = @"\d+" }  
);
```

Binding, Validation, Routing

- **Attribute**

- Aufruf von `config.MapHttpAttributeRoutes()`:
// Selektion nach Kategorie Name, :alpha -> string!
// api/category/Datenzugriff/products
[ResponseType(typeof(Models.Product))]
[Route("api/category/{name:alpha}/products")]
public async Task<IHttpActionResult> GetCategory(string name)
{
 using (var db = new ProductsContext())
 {
 return Ok(await db.Products.Where(t => t.Category.Name == name)
 .Include(t => t.Category)
 .ToListAsync());
 }
}

Validation

Erweiterte Techniken



Binding, Validation, Routing

- **Validierung von eingehenden Daten**

- Mittels Attributen im Model z.B.:
 - [Required]
 - [MinLength(10)]
 - [MaxLength(255)]
 - [Range(1, int.MaxValue)]
 - ...
- Datenbank Updaten, um Exceptiones zu vermeiden!
- PMC:
Add-Migration ConstraintsErstellt,
Update-Database



Binding, Validation, Routing

- Controller, POST Methode anpassen
- Z.B.: HTTP Status Code 400 (Bad Request) zurück geben

```
// Validierung
if (!ModelState.IsValid)
{
    return BadRequest(ModelState);
}
```



Security

Erweiterte Techniken



Security

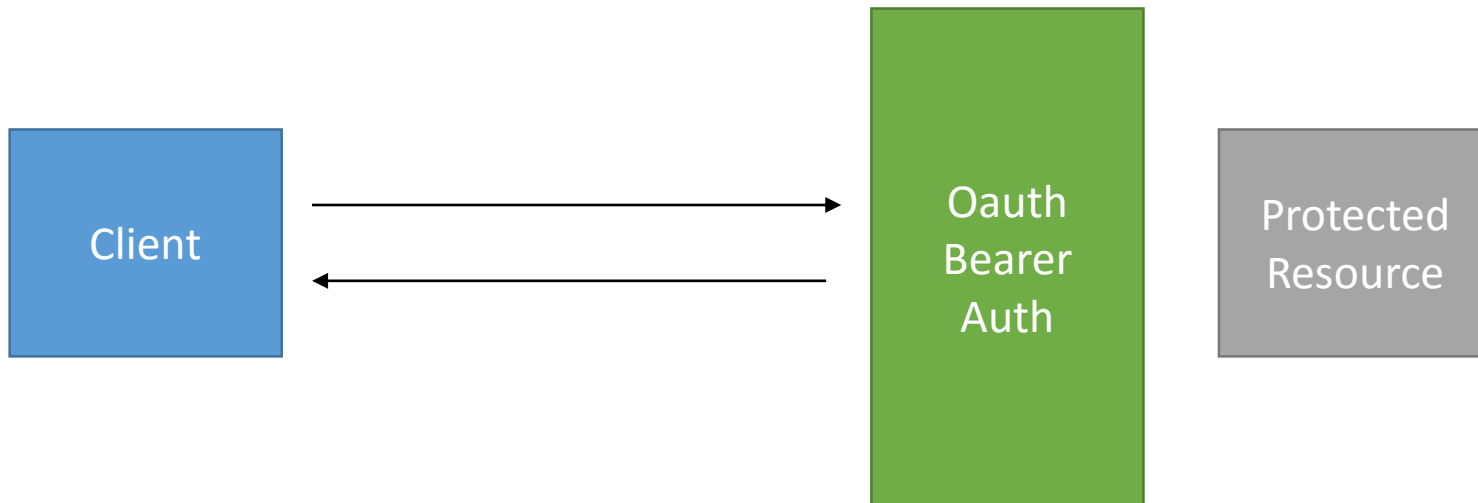
- **Unauthorisierten Zugriff auf die Web API unterbinden**
 - Verwendung von Authorization Filtern (z.B. [Authorize])
 - Implementierung der Authorization Logik
 - AuthorizationFilterAttribute oder Interface IAuthorizationFilter
 - Zuweisung: Global, am Controller oder auf der Action



Security

- **Oauth 2.0**

- Requests mit Oauth 2.0 Bearer Token absichern
- Bearer Token in Standard Authorization Header einbinden
- Middleware validiert und konvertiert „Token“ in „Claims“
- Voraussetzung: SSL Verbindung



Security

- **Tokens von Trusted Authorization Servern beziehen**
 - **Validierungsserver selbst hosten**
 - Eingebaute MS OWIN Komponenten zur Authorisierung
 - Verwenden des ThinkTecture AuthorizationServer
 - **Bestehende Lösungen Verwenden**
 - Windows Azure Active Directory
 - Active Directory Federation Services in Windows Server 2012 R2
 - Windows Azure Mobile Services



Security

- **Authentifizierung und Autorisierung**

- Attribut **[Authorize]**
- Authentifizierung mit OWIN (Middleware)
- Authentifizierung z.B. für POST, PUT, DELETE gut geeignet
- Authentifizierung über Nuget Pakete:
 - Microsoft.AspNet.Identity.Owin
 - Microsoft.AspNet.Identity.Entity
 - Microsoft.owin.host.systemweb



Security

- **CORS (Cross-Origin Resource Sharing)**

- Keine Domänen-übergreifende AJAX Datenübertragung erlaubt
- Cross-Origin Ressource Sharing aktivieren!
- Aktivieren: NuGet-Paket und Sourcecode:
 - Suche cors: Microsoft ASP.NET Web API 2.1 Cross-Origin Support
 - App_Start → WebApiConfig.cs → `config.EnableCors();`
 - Attribut über Klasse oder Methode

```
[EnableCors(origins: "*", headers: "*", methods: "*")]
```

OData

Open Data Protocol, Version 4
„SQL Light für das WEB“

OData

- **Aktuelle Problematik BIG DATA:**

- Eine zunehmende Anzahl an Informationen muss von einer steigenden Anzahl an unterschiedlichen Datenquellen auf den verschiedensten Geräten ganzheitlich bezogen werden können.

- **Einheitliches Web Protokoll - die Lösung**

- Ein Web Protokoll zum Abfragen, bearbeiten und hinzufügen von Daten auf Basis von Web Technologien, wie HTTP, XML und JSON

OData

- **Das Web Protokoll OData – Open Data Protokoll**

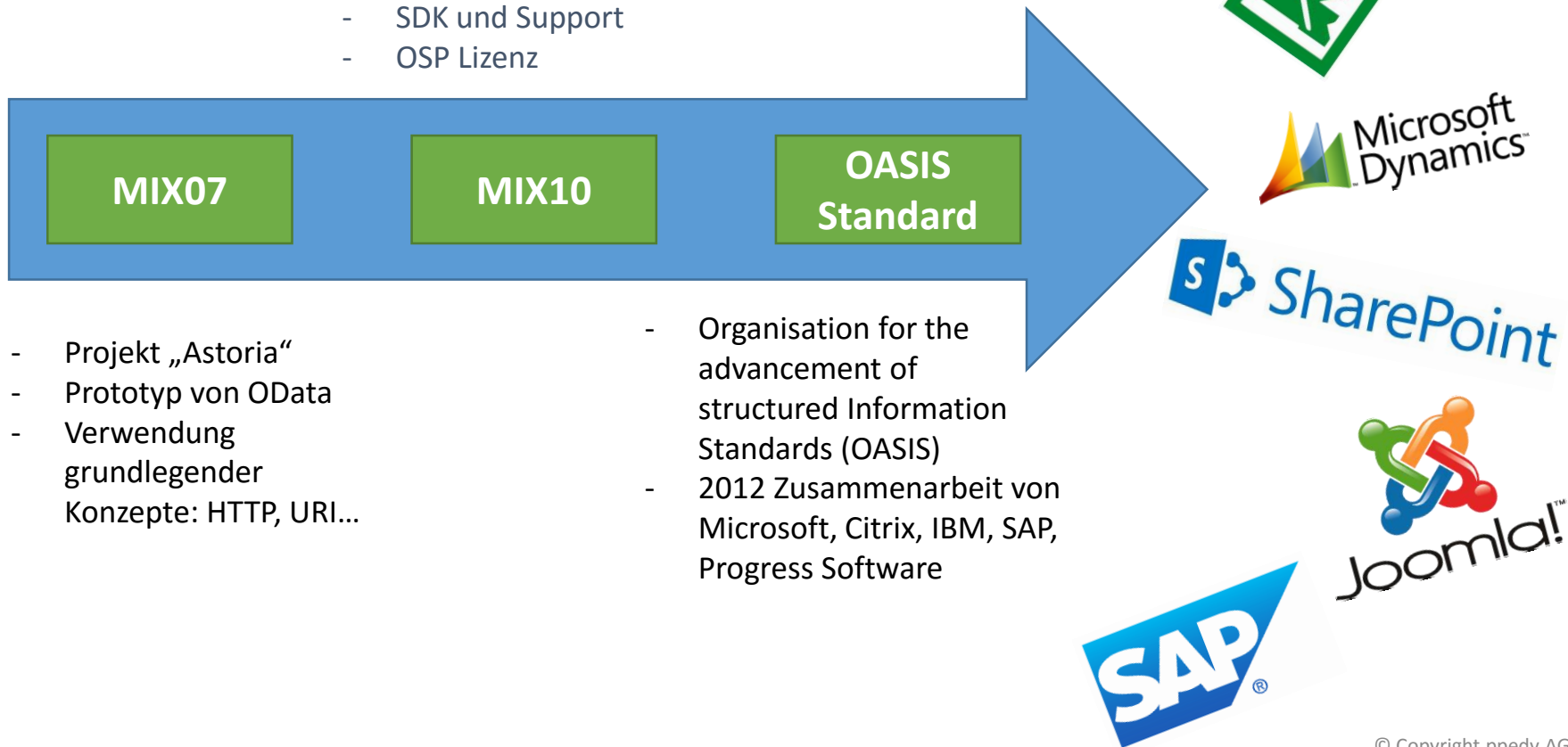
- REST basiertes, offenes Web Protokoll von Microsoft
- Ziel: Ressourcen über das HTTP Protokoll zur Verfügung stellen
- Basiert auf bekannten Standards: AtomPub, XML, JSON
- Mächtige Abfragesprache mit einer Vielzahl an intuitiven URI Konventionen für Navigation, Filterung, Sortierung, Paging
- Uneingeschränkte Verwendung durch Microsoft Open Specification Promise (OSP) gewährleistet
- <http://www.odata.org/>



OData

• Die Entstehung von OData, V 4

- OData offiziell
- Open Data Philosophie
- SDK und Support
- OSP Lizenz



OData

- **4 Bausteine einer OData Implementierung**

- **OData Datenmodell**

- Generische Organisation und Beschreibung der Daten

- **OData Protokoll**

- Ermöglicht HTTP basierte RESTful CRUD Client – Server Interaktion auf Basis einer OData spezifischen Abfragesprache

- **OData Client Bibliotheken**

- Erleichtern die Erstellung von Software, die auf Daten mithilfe des OData Protokolls zugreifen.

- **OData Service**

- Ein abstrahiertes OData Datenmodell, ermöglicht es Daten in einem für den Client lesbaren Format darzustellen

OData

- **2 Bausteine eines OData Service**

- **Service Dokument**

- Darstellung sämtlicher Ressourcen eines Service, auf die über eine URI, einen Namen oder verschiedene Operationen zugegriffen werden kann.

- **Service Metadata Dokument**

- Darstellung sämtlicher Metadaten eines Services, wie Modell, Typen, Aktionen, Beziehungen und detaillierte Informationen über die Semantik des Modells



OData

- **Elemente eines OData Dokuments – EDM Konzept (1)**

- **Entity Type**

- Beschreibt eine Collection und hat einen Namen.

- **Entity**

- Instanzen eines Entity Types bestehend aus Schlüsselfeld(ern) und Eigenschaften z.B. Kunde, Mitarbeiter, ...

- **Entity Set**

- Entities sind in Entity Sets gruppiert z.B. Kunden als Entity Set des Entity Types Kunde.

OData

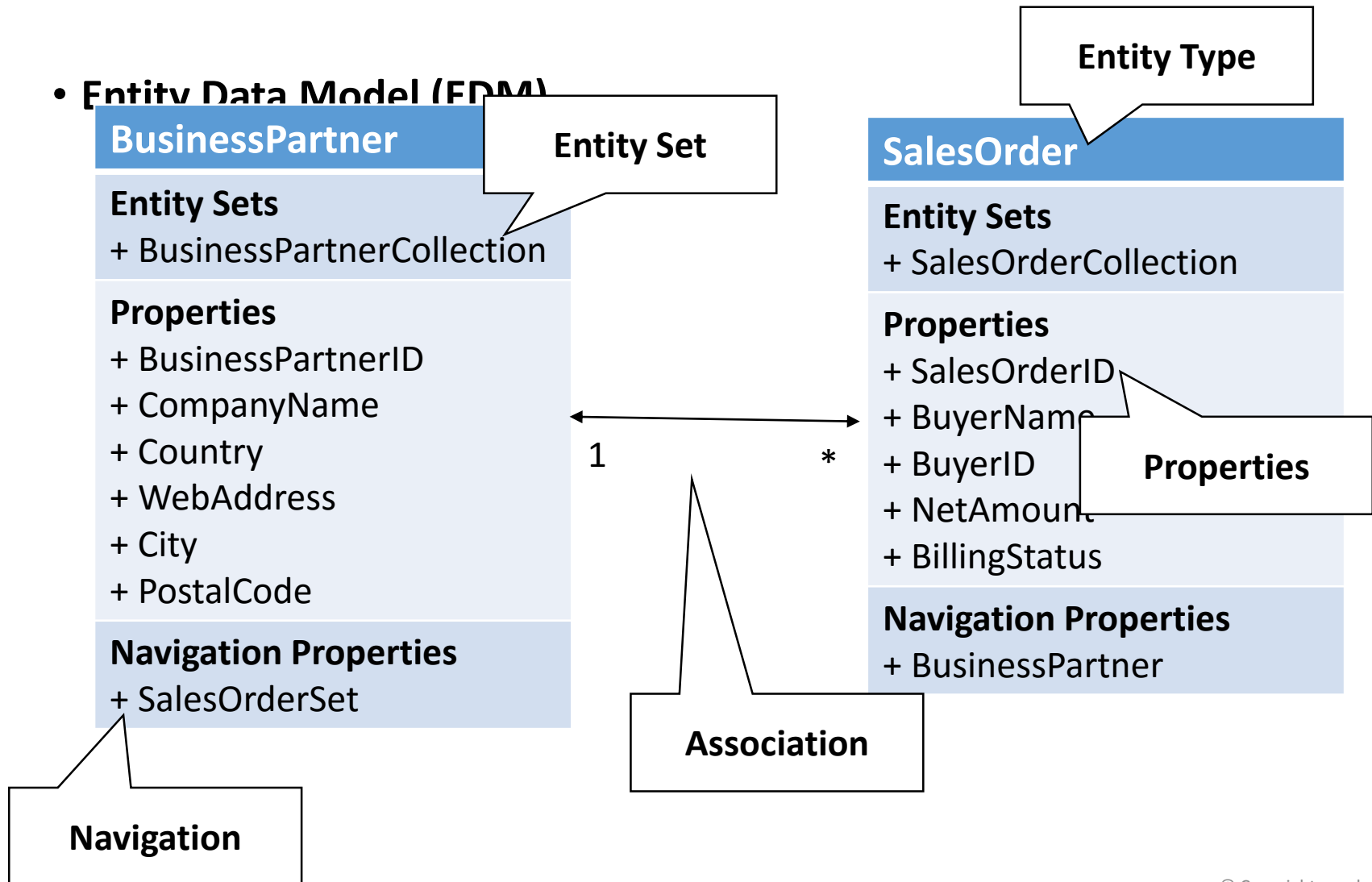
- **Elemente eines OData Dokuments – EDM Konzept (2)**

- **Property**
Einfaches Datenelement, Strukturierte Datenmenge oder Link auf andere Ressource.
- **Navigation property**
Zuständig für Links zwischen Entities.
- **Association**
Beschreibt die Beziehung zwischen Entity Types.



OData

• Entity Data Model (EDM)



OData

- **OData und ASP.NET Web API**

- Offenes und standardisiertes HTTP-basiertes Protokoll für die Interaktion mit REST-basierten Daten und Services
- Löst gängige Probleme (Query, Batching, Paging, Relationships, Metadata, etc.)
- Unterstützung von einer Vielzahl an Clients, Services und Frameworks



OData

- **OData in ASP.NET**
- Komponenten zur Erstellung von OData Services
 - Model Builder, Formatter, Path und Query Parser, Expression Generatoren
 - Individuell anpassbare Services
- OData Service in 3 Schritten
 - Model Definieren
 - OData Route hinzufügen
 - Entity Sets implementieren



OData

- http://localhost:1656/odata/\$metadata#Posts

GET

URL params

Send

Add to collection

Body

Headers (11)

STATUS 200 OK

TIME 19 ms

Pretty

Raw

Preview

JSON

XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <edmx:Edmx Version="1.0"
3   xmlns:edmx="http://schemas.microsoft.com/ado/2007/06/edmx">
4   <edmx:DataServices m:DataServiceVersion="3.0" m:MaxDataServiceVersion="3.0"
5     xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
6     <Schema Namespace="ODataSamples.Controllers"
7       xmlns="http://schemas.microsoft.com/ado/2009/11/edm">
8       <EntityType Name="Post">
9         <key>
10          <PropertyRef Name="Id" />
11        </key>
12        <Property Name="Id" Type="Edm.Int32" Nullable="false" />
13        <Property Name="Title" Type="Edm.String" />
14        <Property Name="Author" Type="Edm.String" />
15        <Property Name="PublishingDate" Type="Edm.DateTime" Nullable="false" />
16      </EntityType>
17    </Schema>
18    <Schema Namespace="Default"
19      xmlns="http://schemas.microsoft.com/ado/2009/11/edm">
20      <EntityContainer Name="Container" m:IsDefaultEntityContainer="true">
21        <EntitySet Name="Posts" EntityType="ODataSamples.Controllers.Post" />
22      </EntityContainer>
23    </Schema>
24  </edmx:DataServices>
25 </edmx:Edmx>

```

OData

• **Se** | <http://localhost:1656/odata/Posts> GET [URL params](#)

[Send](#) [Add to collection](#)

Body Headers (11) **STATUS** 200 OK **TIME** 41 ms

[Pretty](#) [Raw](#) [Preview](#) [JSON](#) [XML](#)

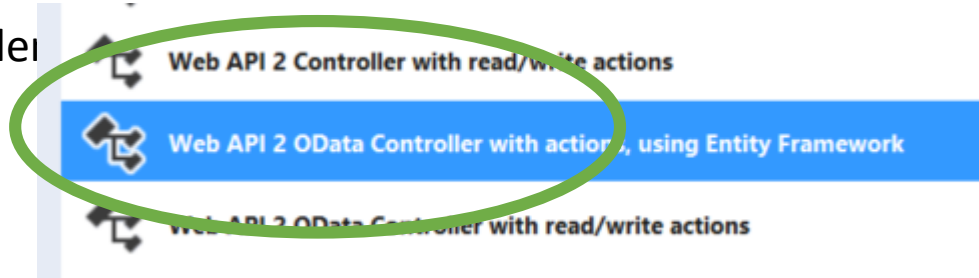
```

1 {
2   "odata.metadata": "http://localhost:1656/odata/$metadata#Posts",
3   "value": [
4     {
5       "Id": 1,
6       "Title": "post1",
7       "Author": "ema",
8       "PublishingDate": "2013-04-05T00:00:00+02:00"
9     },
10    {
11      "Id": 2,
12      "Title": "post2",
13      "Author": "ale",
14      "PublishingDate": "2013-04-06T00:00:00+02:00"
15    },
16    {
17      "Id": 3,
18      "Title": "post3",
19      "Author": "vale",
20      "PublishingDate": "2013-04-03T00:00:00+02:00"
21    },
22    {
23      "Id": 4,
24      "Title": "post4",
25      "Author": "lory",
26      "PublishingDate": "2013-04-06T00:00:00+02:00"
27    },
28    {
29      "Id": 5,
30      "Title": "post5",
31      "Author": "tara"
  
```


OData

- **OData API Controller**

- Implementations



- ASP.NET Web API unterstützt OData
- Voller Support ab Visual Studio 2012 Update 2, oder
- Microsoft ASP.NET Web API OData NuGet Packet

OData

- **Route** in App_Start/WebApiConfig.cs

```
// OData
```

```
ODataModelBuilder builder = new  
ODataConventionModelBuilder();  
builder.EntitySet<Categories>("Categories1");
```

```
Microsoft.Data.Edm.IEdmModel model = builder.GetEdmModel();  
config.Routes.MapODataRoute(  
    routeName: "ODataRoute",  
    routePrefix: "odata", // null  
    model: model);
```

OData

- **Grundlagen OData Operationen(1)**

- In OData API Controller umsetzen!
- Komplette Übersicht möglicher Operationen in OData V. 4:
 - <http://docs.oasis-open.org/odata/odata/v4.0/odata-v4.0-part1-protocol.html>
- **Metadata:**
Aufruf Service Meta Dokument
`/odata/$metadata`
- **OrderBy:**
Sortierung des Dokuments, mehrere Attribute möglich
`/odata/Posts?$orderby=Author desc`
`/odata/Posts?$orderby=Author,PublishingDate`

OData

- **Grundlagen OData Operationen(2)**

- **Top:**

- Anzahl der zurückgegeben Elemente

- `/odata/Posts?$top=2`

- **Inlinecount:**

- Anzahl der Items in der Collection

- `/odata/Posts?$inlinecount=allpages`

- **Skip:**

- Überspringen der ersten n Einträge

- `/odata/Posts?$skip=2`

OData

- Grundlagen OData Operationen(3)

- Filter:

`/odata/Posts?$filter=Author%20eq%20'ema'`

`/odata/Posts?$filter=Author%20eq%20'ema'%20and%20Title%20eq%20'post2'`

Operator	Beschreibung	C# Equivalent
eq	Equal	==
ne	Not equal	!=
gt	Greater than	>
ge	Greater than or equal	>=
lt	Less than	<
le	Less than or equal	<=
and	And	&&
or	Or	

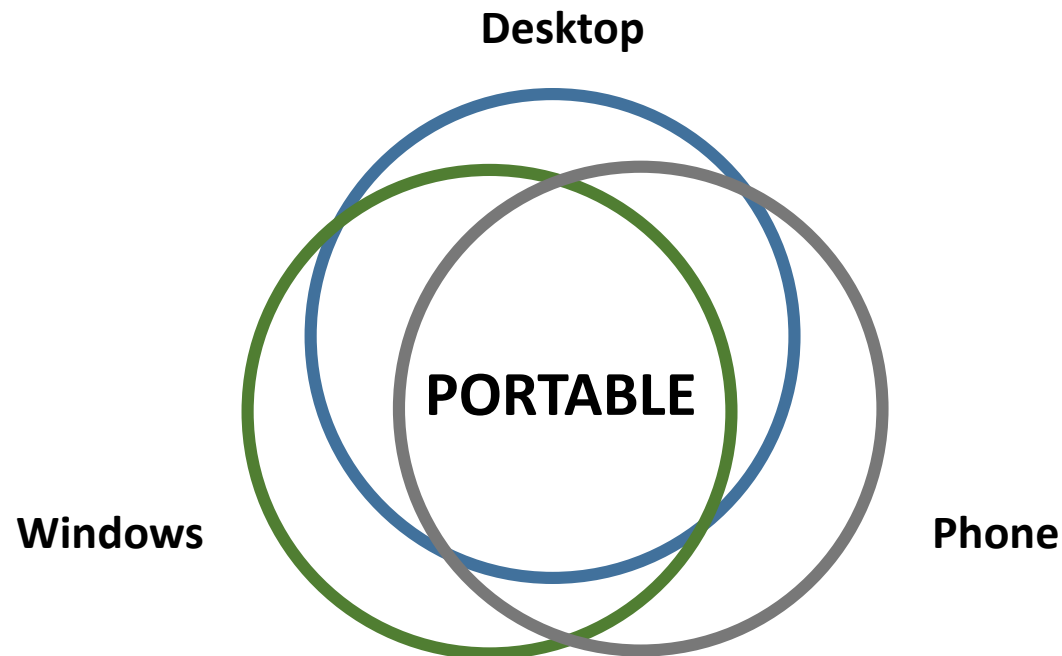
Clients

Für die Web API 2.0



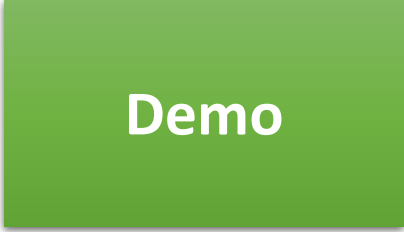
Clients

- Logik in „Portable Class Libraries“



Clients

- **JavaScript-Anwendung**
 - AJAX Call
 - `XMLHttpRequest()` ;
- **WPF-Anwendung**
 - Data Binding



Demo